

Integrasi NodeMCU + Firebase + Sensor DHT11

Praktikum Teknologi IOT

Dedy Yasriady, M.Kom

Presentasi Perkuliahan

Monitoring Temperatur ESP32 ke REST API

Dosen Pengampu: Dedy Yasriady

Mata Kuliah: Internet of Things (IoT)

Tujuan Praktikum

Mahasiswa mampu:

- Memahami konsep IoT berbasis ESP32
- Menghubungkan ESP32 ke jaringan WiFi
- Mengirim data sensor ke server menggunakan HTTP POST
- Memahami format JSON pada komunikasi IoT
- Mengintegrasikan ESP32 dengan REST API

Arsitektur Sistem



Komponen Sistem

Komponen	Fungsi
ESP32	Mikrokontroler utama
Sensor Temperatur	Membaca suhu lingkungan
WiFi	Media komunikasi jaringan
REST API	Endpoint penerima data
Database	Penyimpanan data monitoring

Alur Kerja Sistem

1. ESP32 terkoneksi ke WiFi
 2. Sensor membaca temperatur
 3. Data dikonversi ke format JSON
 4. ESP32 mengirim data menggunakan HTTP POST
 5. Server menerima dan menyimpan data
 6. Proses dilakukan setiap 3 detik
-

Struktur Program

Program terdiri dari:

- Konfigurasi WiFi
 - Konfigurasi API
 - Fungsi pembacaan temperatur
 - HTTP POST ke server
 - Loop pengiriman data berkala
-

Konfigurasi WiFi

```
const char* WIFI_SSID      = "Wokwi-GUEST";  
const char* WIFI_PASSWORD = "";
```

Fungsi:

- Menghubungkan ESP32 ke jaringan internet
 - Menjadi jalur komunikasi ke server IoT
-

Konfigurasi API

```
const char* API_URL  = "https://iotweb.apache.web.id/api/save-data";  
const char* API_TOKEN = "YOUR_TOKEN";
```

Fungsi:

- Menentukan alamat server tujuan
 - Token digunakan untuk autentikasi
-

Pembacaan Sensor Temperatur

```
float readTemperature() {  
    int raw = analogRead(TEMP_PIN);
```

```
float voltage = raw * 3.3 / 4095.0;
float tempC = (voltage - 0.5) * 100.0;

return tempC;
}
```

Konversi ADC ke Temperatur

Persamaan yang digunakan:

$$Voltage = ADC \times \frac{3.3}{4095}$$

$$[\text{Temperature} = (\text{Voltage} - 0.5) \times 100]$$

Format JSON

```
{
  "temperature": 27.5,
  "device_id": "esp32-wokwi-01"
}
```

Keuntungan JSON:

- Ringan
 - Mudah diproses
 - Standar komunikasi web modern
-

HTTP POST

```
int httpCode = http.POST(json);
```

HTTP POST digunakan untuk:

- Mengirim data ke server
 - Menyisipkan data baru ke database
 - Mendukung komunikasi REST API
-

Interval Pengiriman Data

```
delay(3000);
```

Artinya:

- Data dikirim setiap 3 detik
- Sistem melakukan monitoring periodik
- Cocok untuk pemantauan real-time sederhana

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>

// ===== CONFIG =====
const char* WIFI_SSID      = "Wokwi-GUEST";
const char* WIFI_PASSWORD = "";

const char* API_URL      = "https://iotweb.apache.web.id/api/save-data";
const char* API_TOKEN    = "f62504400873ef044d68126c13423223";

const char* DEVICE_ID = "esp32-wokwi-02";

const int TEMP_PIN = 34;
// =====

void setup() {
  Serial.begin(115200);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.print("Menghubungkan WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected");
}

float readTemperature() {
  int raw = analogRead(TEMP_PIN);

  float voltage = raw * 3.3 / 4095.0;
  float tempC = (voltage - 0.5) * 100.0;

  // fallback simulasi Wokwi
  if (tempC < -40 || tempC > 85) {
    tempC = 25 + random(0, 100) / 10.0;
```

```

    }

    return tempC;
}

void loop() {

    float temp = readTemperature();

    Serial.print("Suhu: ");
    Serial.println(temp);

    String json =
        "{\"temperature\":\"" +
        String(temp, 2) +
        "\", \"device_id\":\"" +
        DEVICE_ID +
        "\"}";

    WiFiClientSecure client;
    client.setInsecure();

    HTTPClient http;

    http.begin(client, API_URL);

    http.addHeader("Content-Type", "application/json");
    http.addHeader("Authorization", String("Bearer ") + API_TOKEN);

    int httpCode = http.POST(json);

    Serial.print("HTTP Code: ");
    Serial.println(httpCode);

    if (httpCode > 0) {
        Serial.println(http.getString());
    }

    http.end();

    delay(3000);
}

```

Contoh Output Serial Monitor

```
WiFi Connected  
Suhu: 26.45  
HTTP Code: 201
```

Interpretasi:

- WiFi berhasil terkoneksi
 - Temperatur berhasil dibaca
 - Data berhasil diterima server
-

Kelebihan Sistem

- Implementasi sederhana
 - Biaya murah
 - Mudah dikembangkan
 - Mendukung monitoring real-time
 - Dapat diintegrasikan dengan dashboard web
-

Kekurangan Sistem

- Bergantung pada koneksi internet
 - Belum menggunakan enkripsi sertifikat valid
 - Belum memiliki mekanisme retry kompleks
 - Belum hemat daya
-

Pengembangan Lanjutan

Beberapa pengembangan yang dapat dilakukan:

- Integrasi LoRa
 - Deep Sleep ESP32
 - Dashboard monitoring
 - Penyimpanan cloud
 - MQTT Protocol
 - Multi sensor
 - Alarm suhu otomatis
-

Kesimpulan

- ESP32 dapat digunakan sebagai node IoT sederhana
- HTTP POST mempermudah integrasi dengan server web
- JSON menjadi format pertukaran data yang efektif
- Sistem monitoring real-time dapat dibangun dengan biaya rendah